

Programa de Ingeniería de Sistemas y Computación

Universidad del Quindío

Curso: Programación Avanzada

Guía: 16

Título: MAQUETACIÓN CON BOOTSTRAP

Duración estimada: 240 minutos

Docentes: Christian Andrés Candela

Objetivo

Integrar Bootstrap en el proyecto Angular de solicitudes universitarias para crear maquetaciones responsivas y modernas, comprendiendo los principios fundamentales de CSS y su complemento con el sistema de componentes y signals de Angular 21.

Conceptos Básicos Previos

- **HTML5:** Estructura básica de una página web.
- **CSS3 y selectores:** Hojas de estilo en cascada, clases, IDs y atributos.
- **Box Model:** Margen, borde, padding y contenido.
- **Flexbox y Grid:** Sistemas de disposición de elementos.
- **Angular CLI, Componentes y Signals:** Guías 14 y 15.
- **toSignal()** : Convierte un `Observable` HTTP en un `Signal` — patrón clave de esta guía.

Contextualización Teórica

¿Qué es la Maquetación Web?

Proceso de estructuración visual de los elementos HTML en una interfaz gráfica. Involucra distribución, jerarquía, responsividad y diseño accesible.

◆ CSS: Fundamentos Clave

CSS (Cascading Style Sheets) permite aplicar estilos visuales a los elementos HTML. Sus reglas se aplican a través de **selectores**, que indican a qué elementos del DOM se aplican los estilos.

✅ Tipos de Selectores

1. **Selector de elemento:** Aplica estilos a todos los elementos de un tipo específico.

```
p {  
  font-size: 16px;  
}
```

2. **Selector de clase:** Se aplica a todos los elementos con una clase determinada.

```
.card {  
  background-color: #f8f9fa;  
}
```

Usa el prefijo `.` y puede aplicarse a múltiples elementos.

3. **Selector de ID:** Se aplica a un único elemento con un `id` específico.

```
#encabezado {  
  color: blue;  
}
```

Usa el prefijo `#` y debe ser único por página.

4. **Selector universal:**

```
* {  
  box-sizing: border-box;  
}
```

5. **Selectores descendentes y combinados:**

```
nav ul li a {  
  text-decoration: none;  
}
```

6. Selector de atributo:

```
input[type="email"] {  
  border-color: green;  
}
```

+ Uso de múltiples clases CSS

En HTML, un elemento puede tener más de una clase para combinar estilos:

```
<div class="badge bg-warning text-dark">MEDIA</div>
```

Aquí se aplican `.badge`, `.bg-warning` y `.text-dark` simultáneamente. Esta característica es el fundamento de la filosofía de Bootstrap.

🎯 Buenas prácticas con selectores

- Usar clases (`.clase`) como regla general para estilos reutilizables.
- Evitar el uso excesivo de `id` para estilos.
- En Angular, los estilos en el `.css` del componente tienen **scope automático** — solo afectan al componente donde se definen.

◆ Bootstrap

Framework CSS/JS creado por Twitter (2010) para estandarizar diseños responsivos.

Características clave:

-  **Sistema de grid responsive:** 12 columnas adaptable a dispositivos.
-  **Componentes preconstruidos:** Botones, formularios, cards, navbar, badges, etc.
-  **Utilidades CSS:** Espaciado (`m-`, `p-`), colores (`text-`, `bg-`), flexbox (`d-flex`).
-  **Compatibilidad:** Funciona en todos los navegadores modernos.

Documentación oficial: <https://getbootstrap.com/docs/5.3/>

◆ Sistema Grid de Bootstrap

Divide la pantalla en 12 columnas flexibles que se adaptan al tamaño del dispositivo:

```

<div class="row">
  <div class="col-md-4">Columna 1 (4/12)</div>
  <div class="col-md-8">Columna 2 (8/12)</div>
</div>

```

Prefijo	Breakpoint	Dispositivo
col-	< 576px	Móvil (siempre activo)
col-sm-	≥ 576px	Móvil grande
col-md-	≥ 768px	Tablet
col-lg-	≥ 992px	Desktop

◆ Font Awesome

Biblioteca de iconos vectoriales escalables.

```
<i class="fa-solid fa-user"></i>
```

Sitio oficial: <https://fontawesome.com/icons>

◆ Navbar de Bootstrap — Menú responsivo

El componente `navbar` de Bootstrap implementa un menú de navegación que **colapsa automáticamente en pantallas pequeñas** y muestra un botón de hamburguesa (≡) en su lugar.

```

<!-- navbar-expand-md: el menú se muestra expandido a partir de ≥ 768px -->
<nav class="navbar navbar-expand-md">

  <!-- El botón hamburguesa: solo visible en móvil -->
  <!-- data-bs-toggle y data-bs-target conectan el botón con el div colapsable -->
  <button class="navbar-toggler" type="button"
    data-bs-toggle="collapse"
    data-bs-target="#miMenu">      <!-- apunta al id del div -->
    <span class="navbar-toggler-icon"></span>
  </button>

  <!-- El menú: oculto en móvil (clase collapse), visible en desktop -->
  <div class="collapse navbar-collapse" id="miMenu">
    <ul class="navbar-nav">
      <li class="nav-item"><a class="nav-link" href="#">Inicio</a></li>
    </ul>
  </div>

</nav>

```

El JavaScript de Bootstrap (incluido en `bootstrap.bundle.min.js`) gestiona automáticamente la animación de apertura/cierre. Angular no necesita ningún código TypeScript adicional para esto.

◆ Pipes en Angular — Transformación de datos en la vista

Un **pipe** es una función de transformación que se aplica directamente en la plantilla HTML usando el operador `|`. No modifica el dato original; solo cambia cómo se **muestra**.

```

<!-- SlicePipe: extrae una porción de un string o array -->
{{ solicitud.fechaCreacion | slice:0:10 }}
<!-- "2026-04-30T00:00:00Z" → "2026-04-30" -->

<!-- UpperCasePipe: convierte a mayúsculas -->
{{ solicitud.estado | uppercase }}

<!-- DatePipe: formatea fechas (requiere importar DatePipe) -->
{{ solicitud.fechaCreacion | date:'dd/MM/yyyy' }}

```

Para usar un pipe standalone en un componente Angular 21, hay que importarlo explícitamente:

```
import { SlicePipe } from '@angular/common';

@Component({
  imports: [SlicePipe],    // ← necesario en componentes standalone
})
```

◆ Class Binding — Clases CSS dinámicas en Angular

Angular permite añadir o quitar clases CSS en función de una expresión TypeScript usando la sintaxis `[class.nombre-clase]` :

```
<!-- Añade la clase 'prioridad-alta' solo si la condición es true -->
<div [class.prioridad-alta]="solicitud.prioridad === 'ALTA'"></div>
```

Es equivalente al operador ternario en el atributo `class` , pero más legible:

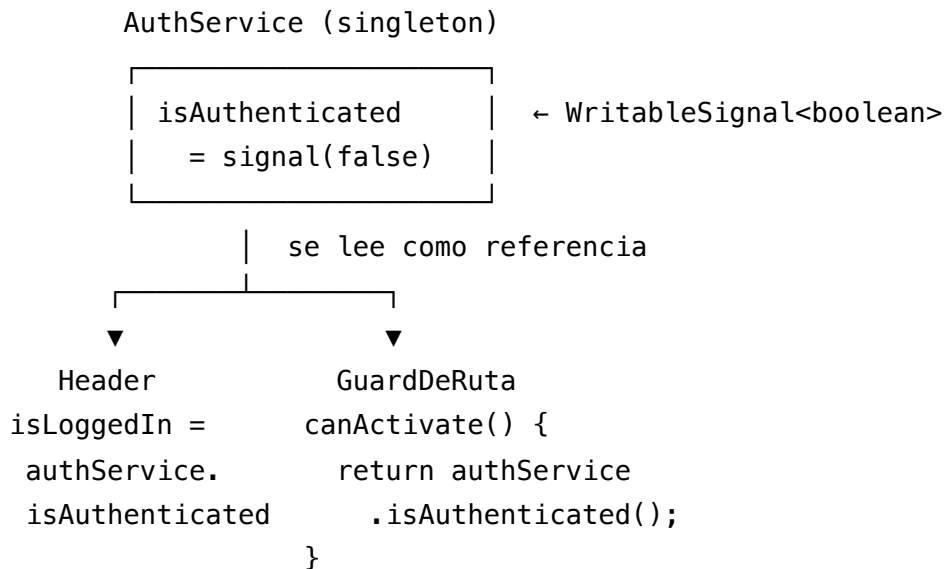
```
<!-- Equivalente menos legible -->
<div class="{{ solicitud.prioridad === 'ALTA' ? 'prioridad-alta' : '' }}"></div>
```

Ambas sintaxis son válidas. `[class.x]` es preferible cuando solo se controla **una clase**.

◆ Estado Compartido con Signals entre Servicio y Componentes

En aplicaciones Angular, el **estado de autenticación** (¿el usuario tiene sesión activa?) es información que varios componentes necesitan al mismo tiempo: el `Header` para mostrar el menú correcto, un `Guard` para proteger rutas, un componente de perfil, etc.

La solución con Signals es elegante: el servicio mantiene **un único signal** y los componentes lo leen directamente:



Cuando el usuario inicia sesión y `isAuthenticated.set(true)` se ejecuta, **todos los componentes que leen ese signal se actualizan automáticamente** sin necesidad de eventos, `EventEmitter`, ni estado duplicado.

```
// En AuthService:  
isAuthenticated = signal(!localStorage.getItem('token'));  
  
logout(): void {  
  localStorage.removeItem('token');  
  this.isAuthenticated.set(false); // ← un cambio, actualización global  
}  
  
// En Header: simplemente se referencia el signal del servicio  
isLoggedIn = this.authService.isAuthenticated;
```

Esta arquitectura se llama **fuentes de verdad única** (*single source of truth*) y es uno de los principios fundamentales del desarrollo de frontend moderno.

◆ toSignal() con HTTP — El patrón de esta guía

En la Guía 15 usamos `toSignal()` para convertir el Observable de estado del formulario en un Signal. En esta guía damos un paso más: **usar `toSignal()` directamente con una petición HTTP** para obtener datos sin escribir `subscribe()`:

```
// ✅ Patrón moderno Angular 21: sin subscribe(), sin gestión manual del Observable
solicitudes = toSignal(
  inject(SolicitudesService).listar(),
  { initialValue: [] as SolicitudResumen[] }
);
```

El array de solicitudes estará disponible como `solicitudes()` en la plantilla desde el primer render. Angular gestiona la subscripción y su cancelación automáticamente.

⚠️ Precauciones y Recomendaciones

1. **NodeJS:** Verifica que tienes instalado NodeJS en su última versión.
2. **Extensión VSCode:** Instala "Bootstrap 5 & Font Awesome Snippets".
3. **Responsive Design:** Usa clases como `col-md-6` para adaptabilidad.
4. **Documentación:** Consulta frecuentemente:
 - [Componentes Bootstrap](#)
 - [Utilidades Bootstrap](#)

📦 Artefactos Requeridos

- Visual Studio Code (última versión) con plugin Angular Language Service.
- Controlador de versiones GIT.
- Node.js y Angular CLI instalados.
- Proyecto Angular de la Guía 15 funcionando.
- Backend Spring Boot corriendo en `http://localhost:8080`.

🏁 Evaluación o Resultado Esperado

Se espera que el estudiante:

1. Instale y configure Bootstrap 5 y Font Awesome en el proyecto Angular.
2. Construya un `Header` responsivo con navegación condicionada al estado de autenticación.

3. Implemente el listado de solicitudes con Bootstrap Cards usando el patrón `toSignal()` con HTTP.
4. Aplique clases CSS personalizadas y de Bootstrap de forma combinada.
5. Comprenda la diferencia entre estilos globales (`styles.css`) y estilos con scope de componente.

Procedimiento

1. Configuración CSS personalizada

Antes de añadir Bootstrap, comprendamos los selectores CSS con un componente propio del proyecto.

1. Genera el componente de listado de solicitudes:

```
ng generate component componentes/lista-solicitudes --skip-tests
```

2. Crea la estructura inicial de `lista-solicitudes.ts` :

En Angular 21, el componente debe existir antes de que su CSS y HTML tengan sentido. La versión completa se implementará en el paso 4.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-lista-solicitudes',
  imports: [],
  templateUrl: './lista-solicitudes.html',
  styleUrls: ['./lista-solicitudes.css']
})
export class ListaSolicitudes {}
```

3. Edita `lista-solicitudes.css` para practicar los distintos tipos de selectores:

```
/* Selector de ID – título principal de la sección */
#titulo-listado {
  color: #1a237e;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

/* Selector de clase – tarjeta base */
.solicitud-card {
  transition: box-shadow 0.2s ease;
}

/* Combinación de clases – tarjeta en estado crítico */
.solicitud-card.prioridad-alta {
  border-left: 4px solid #dc3545;
}

/* Selector de atributo – inputs del formulario */
input[type="text"],
input[type="email"] {
  border-radius: 6px;
}
```

3. Edita `lista-solicitudes.html` con una estructura HTML básica (sin Bootstrap aún):

```
<h1 id="titulo-listado">Mis Solicitudes</h1>
<div class="solicitud-card">
  <p>Aquí aparecerán las solicitudes...</p>
</div>
```

4. Ejecuta la aplicación y observa los estilos aplicados:

```
npm start
```

2. 🛠 Bootstrap — Configuración Inicial

1. Instala las dependencias:

```
npm install bootstrap@5.3.6 @fortawesome/fontawesome-free
```

2. Configura `angular.json` para incluir los estilos y scripts globales:

```
"styles": [  
  "node_modules/bootstrap/dist/css/bootstrap.min.css",  
  "node_modules/@fortawesome/fontawesome-free/css/all.min.css",  
  "src/styles.css"  
],  
"scripts": [  
  "node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"  
]
```

Después de modificar `angular.json`, reinicia el servidor de desarrollo (`Ctrl+C` y `npm start`) para que los cambios surtan efecto.

3. 🧱 Construcción de Componentes de Layout

3.1 🧩 Componente Header

```
ng generate component componentes/header --skip-tests
```

El header debe adaptarse al estado de autenticación: mostrar **Login/Registro** si no hay sesión, y el menú real si el usuario ya está autenticado. Para esto usamos el `AuthService` de la Guía 15.

header.ts :

```

import { Component, inject } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { RouterLink, RouterLinkActive } from '@angular/router';
import { AuthService } from '../../servicios/auth';
import { Router } from '@angular/router';

@Component({
  selector: 'app-header',
  imports: [RouterLink, RouterLinkActive],
  templateUrl: './header.html',
  styleUrls: ['./header.css']
})
export class Header {
  private authService = inject(AuthService);
  private router = inject(Router);

  // Signal que refleja si el usuario tiene sesión activa
  isLoggedIn = this.authService.isAuthenticated;

  cerrarSesion(): void {
    this.authService.logout();
    this.router.navigate(['/login']);
  }
}

```

Para que esto funcione, agrega el método `logout()` y convierte `isAuthenticated` en un signal en `AuthService`:

Actualiza `src/app/servicios/auth.ts` añadiendo:

```

// Signal de autenticación – toda la app puede leerlo
isLoggedIn = signal(!!localStorage.getItem('token'));

logout(): void {
  localStorage.removeItem('token');
  localStorage.removeItem('roles');
  this.isAuthenticated.set(false);
}

```

Y actualiza el método `login()` para que sincronice el signal al autenticarse exitosamente (esto se hace en el `next` del `subscribe` en `login.ts`, agregando

```
this.authService.isAuthenticated.set(true) después de localStorage.setItem ).
```

header.html :

```

<nav class="navbar navbar-expand-md bg-body-tertiary shadow-sm">
  <div class="container">
    <!-- Logo y título -->
    <a routerLink="/" class="navbar-brand d-flex align-items-center gap-2">
      <i class="fa-solid fa-file-alt"></i>
      <span>Solicitudes UQ</span>
    </a>

    <!-- Botón colapso móvil -->
    <button class="navbar-toggler" type="button"
      data-bs-toggle="collapse" data-bs-target="#navMenu">
      <span class="navbar-toggler-icon"></span>
    </button>

    <!-- Menú -->
    <div class="collapse navbar-collapse" id="navMenu">
      <ul class="navbar-nav me-auto">
        <li class="nav-item">
          <a routerLink="/" routerLinkActive="active" class="nav-link">
            <i class="fa-solid fa-home me-1"></i>Inicio
          </a>
        </li>
        @if (isLoggedIn()) {
          <li class="nav-item">
            <a routerLink="/lista-solicitudes" routerLinkActive="active" class="nav-link">
              <i class="fa-solid fa-list me-1"></i>Mis Solicitudes
            </a>
          </li>
          <li class="nav-item">
            <a routerLink="/nueva-solicitud" routerLinkActive="active" class="nav-link">
              <i class="fa-solid fa-plus me-1"></i>Nueva Solicitud
            </a>
          </li>
        }
      </ul>

      <!-- Botones de sesión -->
      <div class="d-flex gap-2">
        @if (isLoggedIn()) {
          <button class="btn btn-outline-danger btn-sm" (click)="cerrarSesion()">
            <i class="fa-solid fa-right-from-bracket me-1"></i>Cerrar Sesión
          </button>
        } @else {

```

```

        <a routerLink="/login" class="btn btn-outline-primary btn-sm">
          <i class="fa-solid fa-right-to-bracket me-1"></i>Ingresar
        </a>
      }
    </div>
  </div>
</div>
</nav>

```

3.2 Componente Footer

ng generate component componentes/footer --skip-tests

footer.html :

```

<footer class="bg-light py-4 mt-5 border-top">
  <div class="container text-center">
    <p class="mb-1 text-muted">
      <i class="fa-solid fa-university me-1"></i>
      © Universidad del Quindío – Programación Avanzada
    </p>
    <div class="social-icons">
      <a href="#" class="text-muted mx-2"><i class="fab fa-instagram"></i></a>
      <a href="#" class="text-muted mx-2"><i class="fab fa-facebook"></i></a>
      <a href="#" class="text-muted mx-2"><i class="fab fa-x-twitter"></i></a>
    </div>
  </div>
</footer>

```

3.3 Integración en app.html

Borra todo el contenido de app.html y reemplázalo por:

```

<app-header></app-header>
<router-outlet></router-outlet>
@defer (on viewport) {
  <app-footer></app-footer>
}

```

¿Qué es @defer ? Es una directiva de carga diferida nativa de Angular 21. El bloque `@defer (on viewport)` indica que el componente `<app-footer>` **no se cargará ni renderizará hasta que el usuario haga scroll y el área del footer entre en el viewport del navegador**. Esto mejora el tiempo de carga inicial de la página (LCP — Largest Contentful Paint) porque Angular no descarga ni procesa el JavaScript del footer hasta que realmente se necesita. Es especialmente útil para componentes en la parte inferior de la página que el usuario puede no llegar a ver.

Luego modifica `app.ts` para añadir `Header` y `Footer` en el arreglo `imports` :

```
import { Header } from './componentes/header/header';
import { Footer } from './componentes/footer/footer';

@Component({
  ...
  imports: [RouterOutlet, Header, Footer],
})
export class App {}
```

3.4 Añade la nueva ruta en `app.routes.ts`

```
import { ListaSolicitudes } from './componentes/lista-solicitudes/lista-solicitudes';

export const routes: Routes = [
  { path: '', component: Inicio },
  { path: 'login', component: Login },
  { path: 'nueva-solicitud', component: NuevaSolicitud },
  { path: 'lista-solicitudes', component: ListaSolicitudes },
  { path: '**', pathMatch: 'full', redirectTo: '/' },
];
```

3.5 Ejecuta la aplicación y observa el header responsivo:

```
npm start
```

Prueba redimensionar la ventana del navegador para ver cómo el menú colapsa en móvil gracias al breakpoint `navbar-expand-md` .

4. Listado de Solicitudes con Bootstrap

Este componente muestra las solicitudes del usuario autenticado usando el `SolicitudesService` creado en la Guía 15. Introduce el patrón `toSignal()` con HTTP: **cero** **subscribe()** .

4.1 Implementa lista-solicitudes.ts

```
import { Component, inject } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { RouterLink } from '@angular/router';
import { SolicitudesService } from '../../servicios/solicitudes';
import { SolicitudResumen } from '../../modelos/solicitudes';

@Component({
  selector: 'app-lista-solicitudes',
  imports: [RouterLink],
  templateUrl: './lista-solicitudes.html',
  styleUrls: ['./lista-solicitudes.css']
})
export class ListaSolicitudes {
  private solicitudesService = inject(SolicitudesService);

  // ✅ toSignal() con HTTP: sin subscribe(), sin takeUntilDestroyed manual
  // Angular gestiona la suscripción automáticamente en el contexto de inyección.
  solicitudes = toSignal(
    this.solicitudesService.listar(),
    { initialValue: [] as SolicitudResumen[] }
  );

  // Mapeo de estado a clase de badge Bootstrap
  badgeEstado(estado: string): string {
    const mapa: Record<string, string> = {
      REGISTRADA: 'bg-secondary',
      CLASIFICADA: 'bg-info text-dark',
      EN_ATENCION: 'bg-primary',
      ATENDIDA: 'bg-success',
      CERRADA: 'bg-dark',
      CANCELADA: 'bg-danger',
    };
    return mapa[estado] ?? 'bg-secondary';
  }

  // Mapeo de prioridad a clase de badge Bootstrap
  badgePrioridad(prioridad: string): string {
    const mapa: Record<string, string> = {
      BAJA: 'bg-success',
      MEDIA: 'bg-warning text-dark',
      ALTA: 'bg-danger',
    };
  }
```

```
};  
    return mapa[prioridad] ?? 'bg-secondary';  
}  
}
```

¿Por qué no hay `subscribe()` aquí? `toSignal()` recibe el Observable de `listar()` y lo convierte en un Signal de solo lectura. Angular se encarga de la suscripción y la cancela automáticamente cuando el componente es destruido, sin necesidad de `DestroyRef` ni `takeUntilDestroyed`.

4.2 Implementa lista-solicitudes.html

```
<div class="container mt-4">

  <div class="d-flex justify-content-between align-items-center mb-4">
    <h1 id="titulo-listado" class="h3 mb-0">
      <i class="fa-solid fa-list-check me-2"></i>Mis Solicitudes
    </h1>
    <a routerLink="/nueva-solicitud" class="btn btn-primary">
      <i class="fa-solid fa-plus me-1"></i>Nueva Solicitud
    </a>
  </div>

  @if (solicitudes().length === 0) {
    <!-- Estado vacío -->
    <div class="text-center py-5 text-muted">
      <i class="fa-solid fa-inbox fa-3x mb-3"></i>
      <p class="h5">No tienes solicitudes registradas aún.</p>
      <a routerLink="/nueva-solicitud" class="btn btn-outline-primary mt-2">
        Crear mi primera solicitud
      </a>
    </div>
  } @else {
    <!-- Grid de tarjetas -->
    <div class="row g-3">
      @for (solicitud of solicitudes(); track solicitud.id) {
        <div class="col-md-6 col-lg-4">
          <div class="card h-100 solicitud-card"
            [class.prioridad-alta]="solicitud.prioridad === 'ALTA'">
            <div class="card-header d-flex justify-content-between align-items-center">
              <span class="fw-semibold">{{ solicitud.codigo }}</span>
              <span class="badge {{ badgeEstado(solicitud.estado) }}">
                {{ solicitud.estado }}
              </span>
            </div>
            <div class="card-body">
              <p class="text-muted small mb-2">
                <i class="fa-solid fa-tag me-1"></i>{{ solicitud.tipo }}
              </p>
              <p class="text-muted small mb-0">
                <i class="fa-regular fa-calendar me-1"></i>
                {{ solicitud.fechaCreacion | slice:0:10 }}
              </p>
            </div>
          </div>
        </div>
      }
    </div>
  }
```

```

        </div>
        <div class="card-footer d-flex justify-content-between align-items-center">
            <span class="badge {{ badgePrioridad(solicitud.prioridad) }}">
                <i class="fa-solid fa-arrow-up me-1"></i>{{ solicitud.prioridad }}
            </span>
            <small class="text-muted">{{ solicitud.nombrePropietario }}</small>
        </div>
    </div>
</div>
}
</div>
}
</div>

```

El pipe | slice:0:10 extrae los primeros 10 caracteres del string ISO-8601 ("2026-04-30T00:00:00Z") para mostrar solo la fecha. Requiere importar SlicePipe en el componente.

Actualiza las importaciones en lista-solicitudes.ts para añadir SlicePipe :

```

import { SlicePipe } from '@angular/common';

@Component({
    ...
    imports: [RouterLink, SlicePipe],
})

```

4.3 Estilos de validación visual en styles.css

Añade en src/styles.css los estilos globales de validación que se aplicarán a todos los formularios del proyecto (login y nueva solicitud de la Guía 15):

```

/* Validación visual Angular + Bootstrap */
.ng-invalid.ng-touched:not(form) {
  border-color: #dc3545;
  box-shadow: 0 0 0 0.2rem rgba(220, 53, 69, 0.25);
}

.ng-valid.ng-touched:not(form) {
  border-color: #198754;
  box-shadow: 0 0 0 0.2rem rgba(25, 135, 84, 0.25);
}

```

Estas reglas usan las clases que Angular añade automáticamente a los `FormControl` según su estado. No requieren ningún cambio en los componentes.

4.4 Ejecuta la aplicación, inicia sesión y navega a "Mis Solicitudes":

```
npm start
```

1. Abre `http://localhost:4200` .
2. Haz clic en **Ingresar** y usa las credenciales `admin@solicitudes.com` / `admin123` .
3. El header debería cambiar automáticamente para mostrar el menú autenticado.
4. Navega a **Mis Solicitudes** y verifica que las tarjetas muestran los datos reales del backend.

5. 🎨 Maquetando el Formulario de Login con Bootstrap

Ahora que Bootstrap está configurado, mejora la apariencia del componente `Login` de la Guía 15.

1. Reemplaza el contenido de login.html con la versión Bootstrap:

```
<div class="container mt-5">
  <div class="row justify-content-center">
    <div class="col-md-5">
      <div class="card shadow-sm">
        <div class="card-header text-center py-3">
          <h5 class="mb-0">
            <i class="fa-solid fa-right-to-bracket me-2"></i>Iniciar Sesión
          </h5>
        </div>
        <div class="card-body">
          <form [formGroup]="loginForm" (ngSubmit)="onSubmit()">

            <div class="mb-3">
              <label class="form-label">Correo electrónico</label>
              <input type="email" formControlName="username"
                class="form-control" placeholder="correo@ejemplo.com" />
              @if (loginForm.get('username')?.invalid && loginForm.get('username')?.touched
                <div class="text-danger small mt-1">
                  Ingresa un correo electrónico válido.
                </div>
              }
            </div>

            <div class="mb-3">
              <label class="form-label">Contraseña</label>
              <input type="password" formControlName="password"
                class="form-control" placeholder="....." />
              @if (loginForm.get('password')?.invalid && loginForm.get('password')?.touched
                <div class="text-danger small mt-1">
                  La contraseña debe tener al menos 6 caracteres.
                </div>
              }
            </div>

            <div class="d-grid">
              <button type="submit" class="btn btn-primary" [disabled]="!canSubmit()">
                @if (isLoading()) {
                  <span class="spinner-border spinner-border-sm me-2"></span>
                  Ingresando...
                } @else {
                  <i class="fa-solid fa-right-to-bracket me-2"></i>Ingresar
                }
              </button>
            </div>
          </form>
        </div>
      </div>
    </div>
  </div>
</div>
```

```

    }
  </button>
</div>

@if (result()) {
  <div class="alert mt-3"
    [class.alert-success]="result().includes('correctamente')"
    [class.alert-danger]="!result().includes('correctamente')">
    {{ result() }}
  </div>
}

</form>
</div>
</div>
</div>
</div>
</div>

```

2. Ejecuta la aplicación y verifica la apariencia del login:

```
npm start
```

6. Avance del Proyecto Final

1. Aplica Bootstrap al header y footer de tu proyecto final, asegurando que el menú sea responsivo con `navbar-expand-md`.
2. Implementa un listado con Bootstrap Cards para la entidad principal de tu proyecto, usando el patrón `toSignal()` con HTTP.
3. Aplica los estilos de validación visual (`ng-invalid.ng-touched`) a todos los formularios de tu proyecto.
4. Asegúrate de que el header muestre opciones diferentes según el estado de autenticación del usuario.



Recursos para la Próxima Clase

1. Investiga sobre **Guards de ruta** en Angular para proteger rutas que requieren autenticación.
 - Referencia: [Angular Route Guards](#)
2. Explora los **badges, spinners y toasts** de Bootstrap para feedback visual al usuario.
 - Referencia: [Bootstrap Components](#)
3. Revisa **Angular animations** para añadir transiciones suaves entre rutas.
 - Referencia: [Angular Animations](#)



Preguntas para Reflexión

- ¿Qué ventaja proporciona Bootstrap frente al CSS puro para un equipo de desarrollo?
- ¿Por qué usamos `toSignal()` en `ListaSolicitudes` y no `subscribe()`? ¿Qué problema resuelve?
- ¿Qué diferencia hay entre estilos en `styles.css` (global) y estilos en el `.css` del componente (scoped)?
- ¿Por qué el header necesita conocer el estado de autenticación? ¿Dónde debería vivir ese estado?



Referencias Bibliográficas

- [Bootstrap 5.3 — Documentación oficial](#)
- [Bootstrap Grid System](#)
- [Bootstrap Cards](#)
- [Angular toSignal — RxJS Interop](#)
- [Font Awesome Icons](#)
- [MDN Web Docs — CSS Selectores](#)